TRISHA ACADEMY

DevOps Career Track · Student Notes





Kubernetes Class 11

Helm Charts, Kustomize and Cluster Essentials

Student notes - concepts, diagrams, live lab and 20 interview questions

60-90 minutes **Everyday examples** **Diagrams** **Live lab** **20 interview Q&A**

For students of TRISHA ACADEMY · Keep these notes for revision and interviews

Contents

Class 11: Helm charts in depth, Kustomize and cluster essentials . . . 3

11.1 Why Helm? . . . 3
11.2 Using public charts . . . 4
11.3 Anatomy of a chart . . . 5
11.4 The template language . . . 7
11.5 Values for many environments . . . 8
11.6 Dependencies, hooks and tests . . . 9
11.7 Debug, package and share . . . 10
11.8 Helm in CI/CD and GitOps . . . 10
11.9 Kustomize - the template-free alternative . . . 10
11.10 Other important concepts . . . 12
11.11 Live lab: your own Helm chart . . . 14
Class 11 - Top 20 interview questions . . . 16

K8S · CLASS 11 · 60-90 minutes

Helm charts in depth, Kustomize and cluster essentials

By the end you can install, write, template, test, package and roll back your own Helm chart, manage environments with Kustomize, and handle key cluster-admin tasks.

What you will learn today

- Why Helm: the copy-paste YAML problem
- Helm concepts: chart, values, release, revision, repository
- Using public charts: search, install, upgrade, rollback, uninstall
- Anatomy of a chart: Chart.yaml, values.yaml, templates, _helpers.tpl, NOTES.txt
- The template language: objects, functions, pipelines, if/range/with
- Values for many environments and the override order
- Dependencies (subcharts), hooks, tests, packaging and OCI registries
- Helm in CI/CD and GitOps; Kustomize as the alternative
- Other essentials: imagePullPolicy, garbage collection and finalizers, CRDs and Operators, kubeadm, cluster upgrades, etcd backup and restore
- Lab: build the quickbite chart and deploy dev and prod releases

Part	Topic	Time
1	Story, Helm concepts and public charts	15 min
2	Writing a chart: templates and values	25 min
3	Dependencies, hooks, packaging, CI/CD	10 min
4	Kustomize and cluster essentials	15 min
5	Live lab	15 min
6	Recap and interview questions	5 min

11.1 Why Helm?

Ravi's story

QuickBite now runs menu, orders and payment in dev, QA and prod. Each app needs a Deployment, Service, Ingress, ConfigMap, Secret, HPA and PDB - 7 files x 3 apps x 3 environments = 63 YAML files that are 90% identical. Last week someone changed the memory limit in dev but forgot prod, and prod crashed. Ravi's lead said: "Write it once as a **Helm chart**, and keep only the differences per environment."

Helm is the package manager for Kubernetes - what apt is to Ubuntu, npm to Node.js or Maven to Java. It turns a folder of templated YAML into a versioned package that you can install, upgrade and roll back with one command.

Everyday example

Think of a wedding card printer. The design (template) is the same for every family; only the names, date and venue change (values). The printer fills in the blanks and prints the final cards (rendered YAML). If the date changes, you reprint from the same design with a new value - you never redraw the card.

Term	Meaning	Example
Chart	A package: templates + default values + metadata	`quickbite`, `podinfo`, `kube-prometheus-stack`
Values	The settings that fill the templates	`replicaCount: 3`, `image.tag: 1.4.2`
Release	One installed copy of a chart, with a name, in a namespace	`shop-dev`, `shop-prod`
Revision	A numbered version of a release, created on every install/upgrade/rollback	1, 2, 3
Repository	A server or OCI registry where charts are published	Artifact Hub, Docker Hub, ECR, ACR

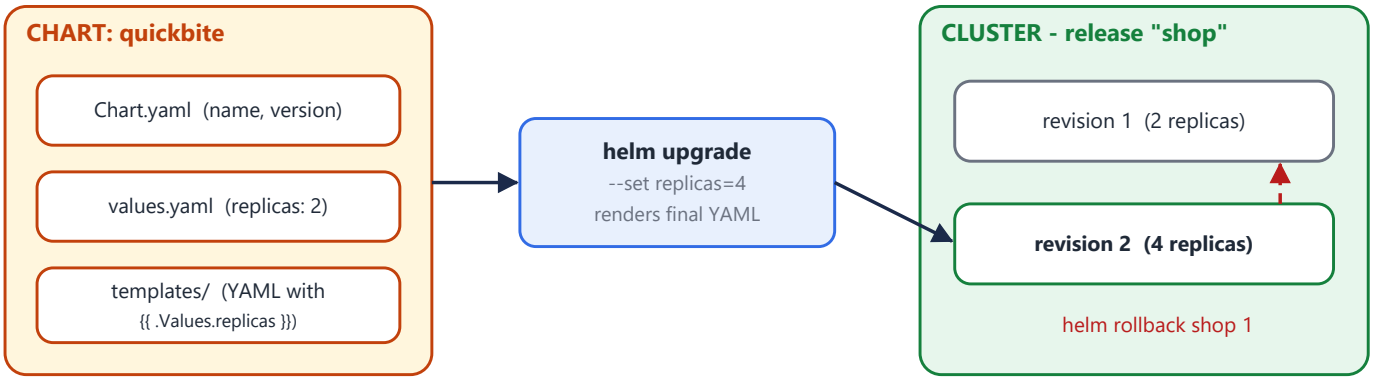


Figure 11.1 - A chart plus values becomes a release with numbered revisions

How Helm works inside

Since Helm 3, Helm is just a client program - there is nothing to install in the cluster (Helm 2's server component, Tiller, was removed for security). Helm renders the templates on your laptop or CI runner, sends the resulting YAML to the API server using your kubeconfig permissions, and stores a record of each revision as a Secret in the release's namespace.

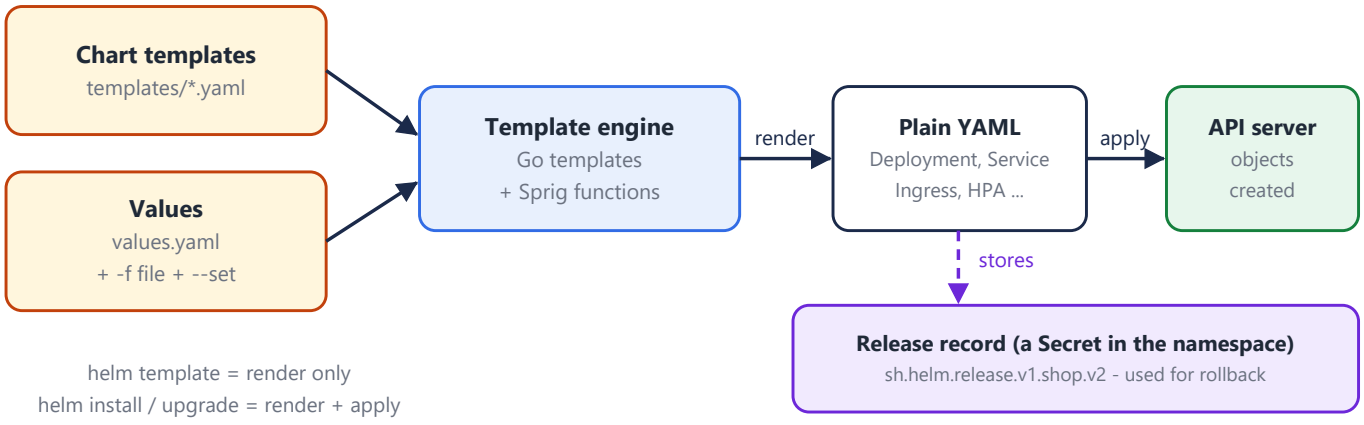


Figure 11.2 - Templates plus values are rendered into plain YAML, applied, and recorded

11.2 Using public charts

Thousands of ready-made charts are listed on **Artifact Hub** (artifacthub.io) - monitoring, databases, ingress controllers, cert-manager, Argo CD and more. We practise with **podinfo**, a small, well-maintained demo app.

PowerShell - public chart

```
helm repo add podinfo https://stefanprodan.github.io/podinfo
helm repo update
helm search repo podinfo
helm show values podinfo/podinfo > podinfo-values.yaml   # every setting you can change

helm install my-app podinfo/podinfo --namespace demo --create-namespace --set replicaCount=2
helm list -n demo
helm status my-app -n demo
kubectl -n demo port-forward svc/my-app-podinfo 9898:9898   # open http://localhost:9898

helm upgrade my-app podinfo/podinfo -n demo --set replicaCount=3 --set ui.message="Hello QuickBite"
helm history my-app -n demo
helm get values my-app -n demo          # the values you supplied
helm get manifest my-app -n demo        # the YAML Helm applied
helm rollback my-app 1 -n demo
helm uninstall my-app -n demo
```

Did you know?
Charts can also live in OCI registries, the same place your images live. Install directly with `helm install my-app oci://ghcr.io/stefanprodan/charts/podinfo` - no `helm repo add` needed.

11.3 Anatomy of a chart

`helm create quickbite` generates a complete, working chart. This is its structure:

Chart folder

```
quickbite/
  Chart.yaml              # name, chart version, appVersion, dependencies
  values.yaml             # default values
  values.schema.json      # optional: validates values (types, required fields)
  charts/                 # downloaded dependency charts (subcharts)
  .helmignore             # files to leave out when packaging
  templates/
    _helpers.tpl          # reusable named templates (names, labels)
    deployment.yaml
    service.yaml
    ingress.yaml
    hpa.yaml
    serviceaccount.yaml
    NOTES.txt             # message printed after install
    tests/
      test-connection.yaml   # run with: helm test <release>
```

Chart.yaml

```
apiVersion: v2                # Helm 3 chart format
name: quickbite
description: QuickBite orders service
type: application             # or 'library' (only helpers, nothing installed)
version: 0.1.0                # CHART version - bump on every chart change (SemVer)
appVersion: "1.4.2"           # version of the APP inside - informational
```

values.yaml

```
replicaCount: 2

image:
  repository: nginx
  tag: "1.27"
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 80

ingress:
  enabled: false
  host: quickbite.local

resources:
  requests: { cpu: 100m, memory: 128Mi }
  limits:   { memory: 256Mi }

env:
  APP_MODE: dev
  LOG_LEVEL: info
```

templates/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "quickbite.fullname" . }}
  labels:
    {{- include "quickbite.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      {{- include "quickbite.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "quickbite.selectorLabels" . | nindent 8 }}
      annotations:
        checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}
    spec:
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion}}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - containerPort: 80
          env:
            {{- range $key, $value := .Values.env }}
            - name: {{ $key }}
              value: {{ $value | quote }}
            {{- end }}
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
```

The `checksum/config` annotation is a well-known trick: whenever the ConfigMap changes, the hash changes, the pod template changes, and the Deployment rolls out new pods automatically. Add that line only once your chart has a `templates/configmap.yaml` - the chart from `helm create` does not include one.

templates/_helpers.tpl

```
{{- define "quickbite.fullname" -}}
{{- printf "%s-%s" .Release.Name .Chart.Name | trunc 63 | trimSuffix "-" }}
{{- end }}

{{- define "quickbite.selectorLabels" -}}
app.kubernetes.io/name: {{ .Chart.Name }}
app.kubernetes.io/instance: {{ .Release.Name }}
{{- end }}

{{- define "quickbite.labels" -}}
{{ include "quickbite.selectorLabels" . }}
helm.sh/chart: {{ .Chart.Name }}-{{ .Chart.Version }}
app.kubernetes.io/version: {{ .Chart.AppVersion | quote }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
{{- end }}
```

templates/ingress.yaml - created only when ingress.enabled is true

```
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ include "quickbite.fullname" . }}
spec:
  rules:
    - host: {{ .Values.ingress.host }}
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: {{ include "quickbite.fullname" . }}
                port: { number: {{ .Values.service.port }} }
{{- end }}
```

11.4 The template language

Helm uses Go templates plus about 70 extra functions from the Sprig library. Everything between `{{ }}` is evaluated; everything else is copied as-is.

Feature	Example	Meaning
Built-in objects	`.Values`, `.Release.Name`, `.Release.Namespace`, `.Chart.Version`, `.Capabilities`, `.Files`	Data available to every template
Pipelines	`{{ .Values.env.mode | upper | quote }}`	Pass a value through functions, left to right
Default	`{{ .Values.image.tag | default "latest" }}`	Fallback when a value is empty
Required	`{{ required "image.tag is required" .Values.image.tag }}`	Fail the install with a clear message
Condition	`{{- if .Values.ingress.enabled }} ... {{- end }}`	Include a block only when true
Loop	`{{- range .Values.hosts }} - {{ . }} {{- end }}`	Repeat for each list item
Scope	`{{- with .Values.nodeSelector }} ... {{- end }}`	Shortcut into a nested value

Feature	Example	Meaning
YAML blocks	`{{- toYaml .Values.resources | nindent 12 }}`	Insert a whole map with correct indentation
Named template	`{{ include "quickbite.labels" . }}`	Reuse a block defined in _helpers.tpl
Whitespace	`{{-` and `-}}`	Trim spaces/newlines left or right of the tag

Common mistake

- Wrong indentation in rendered YAML is the number-one Helm bug. Use `nindent` with the exact column, and always check with `helm template`.
- Forgetting `quote` on strings such as `"true"` or `"1.10"` - YAML turns them into a boolean or the number 1.1.

11.5 Values for many environments

Keep one chart and one small values file per environment. Only the differences go in it:

```
# values-dev.yaml
replicaCount: 1
image:
  tag: "dev-42"
env:
  APP_MODE: dev
  LOG_LEVEL: debug
ingress:
  enabled: false
```

```
# values-prod.yaml
 replicaCount: 3
 image:
   tag: "1.4.2"
 env:
   APP_MODE: prod
   LOG_LEVEL: warn
 ingress:
   enabled: true
   host: orders.quickbite.com
```

```
helm upgrade --install shop-dev  ./quickbite -n dev  --create-namespace -f values-dev.yaml
helm upgrade --install shop-prod ./quickbite -n prod --create-namespace -f values-prod.yaml
```

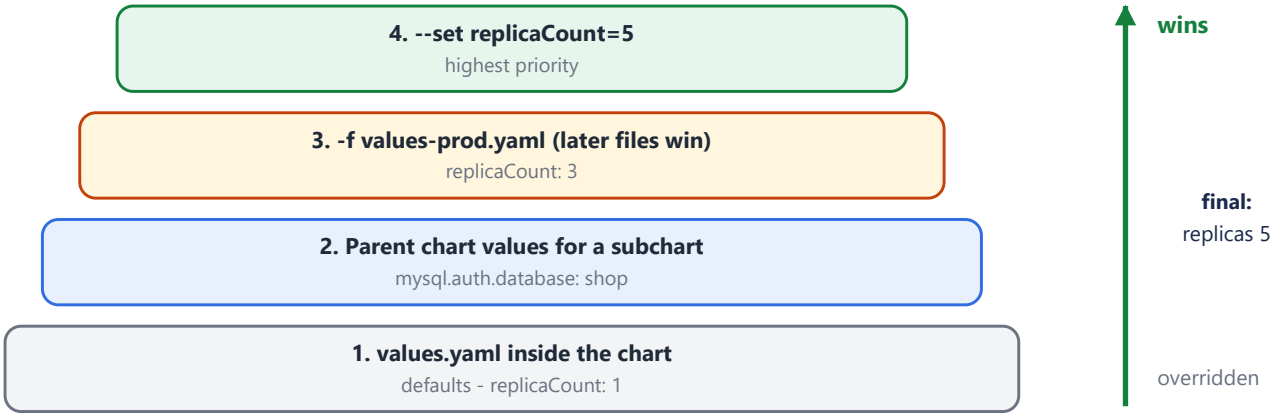


Figure 11.3 - When the same value is set in several places, the higher layer wins

Remember

- Order of priority (lowest to highest): chart `values.yaml` -> parent chart values -> `-f` files (the last file wins) -> `--set` / `--set-string` / `--set-file`.
- Use `-f` files kept in Git for anything permanent; use `--set` only for quick changes or the image tag from CI.
- Never put real passwords in values files in Git - use External Secrets, Sealed Secrets or the helm-secrets plugin with SOPS.

11.6 Dependencies, hooks and tests

Dependencies (subcharts)

A chart can depend on other charts - for example orders needs a Redis cache. Declare it in `Chart.yaml`, then run `helm dependency update` to download it into `charts/`. The parent configures the subchart under a key with the subchart's name; `global:` values are shared with all subcharts.

Chart.yaml + values.yaml

```
dependencies:
  - name: redis
    version: "~20.0.0"                # check the chart's current version on Artifact Hub
    repository: oci://registry-1.docker.io/bitnamicharts
    condition: redis.enabled          # install only when redis.enabled is true

# values.yaml of the parent chart
redis:
  enabled: true
  architecture: standalone
```

Hooks

Hooks run a resource - usually a Job - at a specific moment of the release lifecycle: `pre-install`, `post-install`, `pre-upgrade`, `post-upgrade`, `pre-rollback`, `pre-delete`, `post-delete` and `test`. The classic use is running database migrations before new pods start.

templates/db-migrate-job.yaml

```
apiVersion: batch/v1
kind: Job
metadata:
  name: {{ include "quickbite.fullname" . }}-db-migrate
  annotations:
    "helm.sh/hook": pre-install,pre-upgrade
    "helm.sh/hook-weight": "0"                      # lower weights run first
    "helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
spec:
  backoffLimit: 1
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: migrate
          image: busybox:1.36
          command: ["sh", "-c", "echo running DB migrations for {{ .Chart.AppVersion }}"]
```

Tests

A pod with the annotation `"helm.sh/hook": test` is a chart test. `helm test <release>` runs it after installation; the test passes if the pod exits with code 0. The generated chart already includes `tests/test-connection.yaml`, which runs `wget` against the Service.

11.7 Debug, package and share

Command	Use
`helm lint ./quickbite`	Check the chart for errors and best-practice warnings
`helm template shop ./quickbite -f values-prod.yaml`	Render YAML locally - nothing touches the cluster
`helm install shop ./quickbite --dry-run --debug`	Render and validate against the cluster without installing
`helm diff upgrade shop ./quickbite`	Show what would change (helm-diff plugin) - great in pull requests
`helm package ./quickbite`	Create `quickbite-0.1.0.tgz`
`helm registry login <registry>`	Log in to an OCI registry (Docker Hub, ECR, ACR, GHCR, Harbor)
`helm push quickbite-0.1.0.tgz oci://<registry>/<path>`	Publish the chart
`helm pull oci://<registry>/<path>/quickbite --version 0.1.0`	Download a published chart

Chart version vs appVersion: `version` is the version of the packaging and must change whenever any chart file changes. `appVersion` is the version of the software inside. A chart 0.3.0 might deploy app 1.4.2.

11.8 Helm in CI/CD and GitOps

Typical pipeline step (bash)

```
helm upgrade --install orders ./charts/quickbite \
  --namespace prod --create-namespace \
  -f values-prod.yaml \
  --set image.tag=$GIT_COMMIT_SHA \
  --atomic --wait --timeout 5m          # roll back automatically if pods are not healthy in 5 min
```

- `upgrade --install` works whether or not the release already exists - one command for every run.
- `--wait` waits until pods are Ready; `--atomic` rolls back automatically on failure.
- In GitOps, Argo CD or Flux render the chart themselves from Git with your values files - you never run `helm install` by hand in production. Note that `helm list` will not show Argo CD apps, because Argo CD uses Helm only to render.
- Helmfile manages many releases across environments from one declarative file.

Did you know?
Helm 4 was released in November 2025. Charts written for Helm 3 keep working. The main changes are server-side apply, smarter waiting for resources, WebAssembly plugins, and some renamed flags - for example `--atomic` is now also called `--rollback-on-failure`. Run `helm version` to see which one you have; every command in this class works on both.

11.9 Kustomize - the template-free alternative

Kustomize is built into kubectl. Instead of templates, you keep plain, valid YAML in a **base** folder and small **overlays** that patch it for each environment.

Everyday example
Kustomize is like buying a plain kurta (base) and having the tailor alter only the sleeves for one person and the length for another (overlays). Helm is like a pattern with blanks for measurements - every kurta is cut fresh from the pattern.

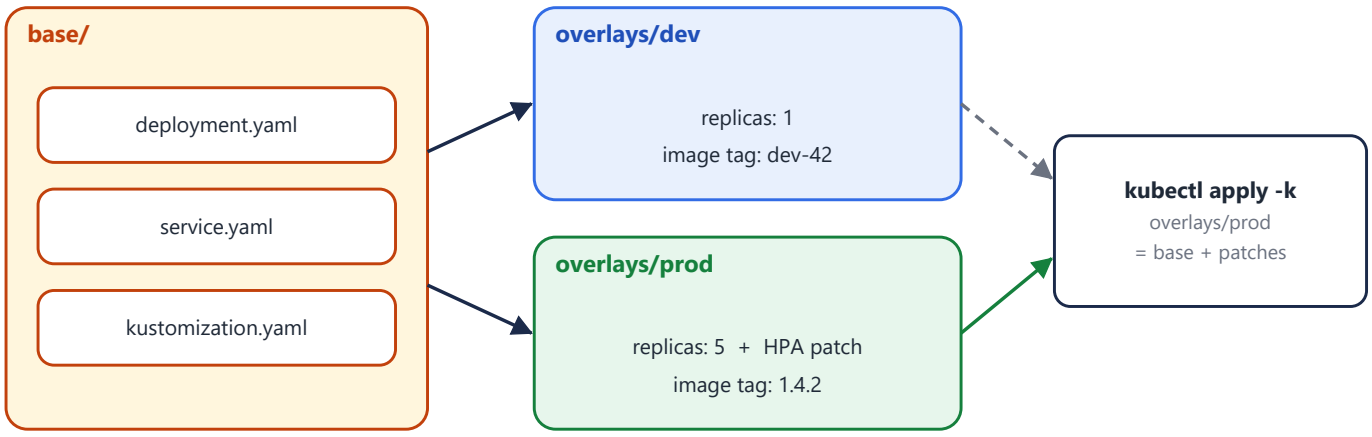


Figure 11.4 - Kustomize overlays patch a shared base for each environment

kustomization files

```
# base/kustomization.yaml
resources:
  - deployment.yaml
  - service.yaml

# overlays/prod/kustomization.yaml
resources:
  - ../../base
namespace: prod
namePrefix: prod-
replicas:
  - name: web
    count: 5
images:
  - name: nginx
    newTag: "1.27"
configMapGenerator:                # adds a hash suffix -> pods restart on change
  - name: app-config
    literals: ["APP_MODE=prod"]
patches:
  - path: memory-patch.yaml
```

```
kubectl kustomize overlays/prod          # preview the final YAML
kubectl apply -k overlays/prod
kubectl delete -k overlays/prod
```

	Helm	Kustomize
Approach	Templates + values	Plain YAML + patches
Installed with	helm CLI	Built into kubectl (`-k`)
Packaging and versions	Yes - charts, repositories, OCI	No - just folders in Git
Release history and rollback	Yes - `helm rollback`	No - use Git revert / GitOps
Hooks and tests	Yes	No

	Helm	Kustomize
Best for	Distributing apps, third-party software, many settings	Your own apps with small per-environment changes

Many teams use both: install third-party software with Helm, and manage their own apps with Kustomize - or even post-process a Helm chart with Kustomize.

11.10 Other important concepts

imagePullPolicy

Value	Behaviour
IfNotPresent	Pull only if the image is not on the node. Default for a fixed tag like `nginx:1.27`.
Always	Check the registry on every container start. Default when the tag is `latest` or missing.
Never	Use only a local image; fail if absent. Handy with `minikube image load`.

Owner references, garbage collection and finalizers

Pods created by a ReplicaSet carry an **ownerReference** pointing to it, and the ReplicaSet points to its Deployment. When an owner is deleted, the **garbage collector** deletes its dependents (cascading deletion). `kubectl delete deploy web --cascade=orphan` keeps the pods instead.

A **finalizer** is a key in `metadata.finalizers` that blocks deletion until a controller finishes cleanup - for example `kubernetes.io/pvc-protection` stops a PVC in use from being deleted. If the controller is gone, the object stays **Terminating** forever; that is why namespaces sometimes get stuck. Removing finalizers by hand is a last resort.

CRDs and Operators

A **CustomResourceDefinition** teaches the API server a new kind of object. An **Operator** is a controller that watches those objects and does what a human expert would: create the StatefulSet, take backups, fail over, upgrade. You install Operators - often with Helm - and then simply write short custom resources.

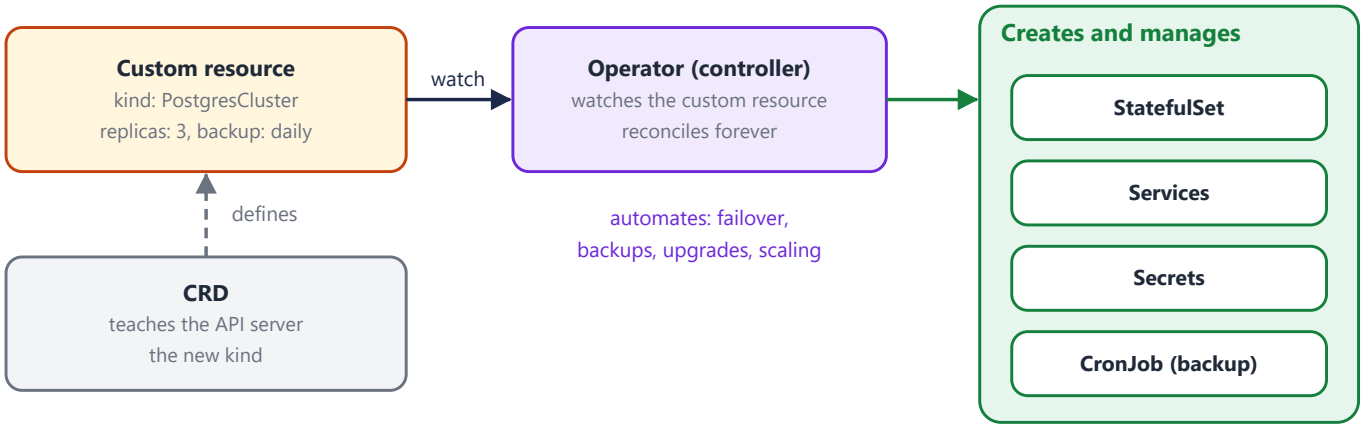


Figure 11.5 - An Operator turns a short custom resource into many managed objects

crd.yaml - try: kubectl apply, kubectl get backups

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: backups.quickbite.io
spec:
  group: quickbite.io
  scope: Namespaced
  names: { kind: Backup, plural: backups, singular: backup }
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                database: { type: string }
                schedule: { type: string }
---
apiVersion: quickbite.io/v1
kind: Backup
metadata: { name: orders-nightly }
spec: { database: orders, schedule: "0 1 * * *" }
```

Without an Operator the Backup object is only stored data; an Operator gives it behaviour. Real examples: cert-manager, Prometheus Operator, Strimzi (Kafka), CloudNativePG (PostgreSQL).

Building a cluster with kubeadm

minikube is for learning; on real Linux VMs the official tool is **kubeadm**. It is also the basis of the CKA exam.

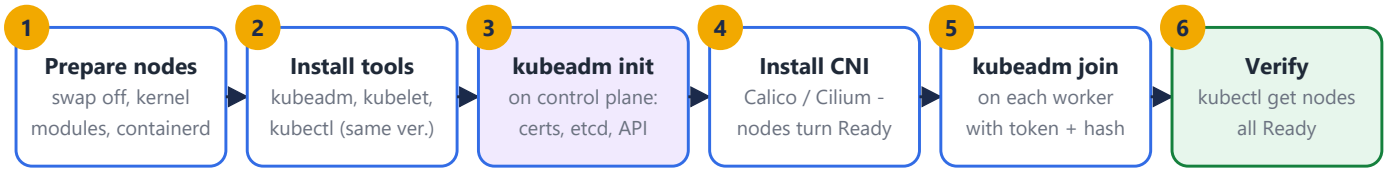


Figure 11.6 - Building a cluster with kubeadm

bash on Linux VMs

```
# on every node (Ubuntu): swap off, containerd with SystemdCgroup = true,
# kubelet + kubeadm + kubectl from pkgs.k8s.io, then: sudo apt-mark hold kubelet kubeadm kubectl
sudo swapoff -a

# control plane
sudo kubeadm init --pod-network-cidr=192.168.0.0/16
mkdir -p $HOME/.kube && sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
kubectl apply -f <calico-manifest-url>         # install the CNI; nodes become Ready
kubeadm token create --print-join-command       # prints the join command

# each worker
sudo kubeadm join 10.0.0.10:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash>
```

Upgrading a cluster

- Upgrade **one minor version at a time** (1.33 to 1.34, never 1.32 to 1.34). Read the release notes for removed APIs first.
- Control plane first: upgrade the kubeadm package, run `kubeadm upgrade plan`, then `sudo kubeadm upgrade apply v1.xx.y`; drain the node, upgrade kubelet and kubectl, restart kubelet, uncordon.
- Then each worker: drain, upgrade kubeadm, `sudo kubeadm upgrade node`, upgrade kubelet, restart, uncordon.
- Version skew rule: the kubelet may be up to three minor versions older than the API server, but never newer.
- On EKS, AKS and GKE the cloud upgrades the control plane; you upgrade node pools.

Backing up and restoring etcd

bash on the control-plane node

```
sudo ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-$(date +%F).db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

sudo etcdutl snapshot status /backup/etcd-2026-10-10.db --write-out=table

# restore into a NEW data directory, then point the etcd static pod to it
sudo etcdutl snapshot restore /backup/etcd-2026-10-10.db --data-dir /var/lib/etcd-restore
# edit the hostPath /var/lib/etcd -> /var/lib/etcd-restore
sudo vi /etc/kubernetes/manifests/etcd.yaml
```

etcd snapshots protect cluster objects only. Application data in volumes needs its own backup - **Velero** backs up namespaces and volume snapshots together, and can restore them into another cluster.

11.11 Live lab: your own Helm chart

PowerShell - part 1

```
helm create quickbite
cd quickbite
#  edit values.yaml: image.repository: nginx, image.tag: "1.26", replicaCount: 1
cd ..
helm lint ./quickbite
helm template shop ./quickbite | more        # read the rendered YAML

helm install shop-dev ./quickbite -n dev --create-namespace
kubectl -n dev get deploy,svc,pods
helm test shop-dev -n dev                    # runs templates/tests/test-connection.yaml
```

PowerShell - part 2

```
# create values-prod.yaml with: replicaCount: 3 and image.tag: "1.27"
helm install shop-prod ./quickbite -n prod --create-namespace -f values-prod.yaml
kubectl -n prod get pods                     # 3 pods on nginx:1.27

# a bad upgrade, then rollback
helm upgrade shop-prod ./quickbite -n prod -f values-prod.yaml --set image.tag=9.99-broken
kubectl -n prod get pods                     # ImagePullBackOff
helm history shop-prod -n prod
helm rollback shop-prod 1 -n prod
kubectl -n prod get secrets -l owner=helm    # the release records Helm keeps

# safer upgrades
helm upgrade shop-prod ./quickbite -n prod -f values-prod.yaml `
  --set image.tag=9.99-broken --atomic --wait --timeout 60s   # fails, rolls back by itself

helm package ./quickbite                     # quickbite-0.1.0.tgz
helm uninstall shop-dev -n dev; helm uninstall shop-prod -n prod
```

Try it yourself

- Add a `configmap.yaml` template driven by `.Values.env`, plus the checksum annotation, and watch pods roll when you change a value.
- Add the db-migrate hook from section 11.6 and watch the Job run before each upgrade: `kubectl -n prod get jobs -w`.
- Convert the same app to Kustomize with `base` and `overlays/dev|prod` folders and compare.

Common mistake

- Changing Helm-managed objects with `kubectl edit` - the next `helm upgrade` overwrites your change. Change the values and upgrade instead.
- Forgetting to bump `version` in Chart.yaml before packaging - registries refuse a duplicate version, or worse, users get stale charts.
- Using `--set` for many values in production - nobody can see them later. Put them in a values file in Git.

Class 11 recap

- Helm = package manager: chart (templates + values) -> release -> revisions.
- Key commands: repo add, search, show values, install, upgrade --install, list, history, rollback, uninstall, template, lint, test, package, push.
- Values priority: chart defaults < parent < -f files < --set.
- Hooks run Jobs at lifecycle moments; `helm test` validates a release.
- Kustomize = base + overlays, built into kubectl, no templates.
- Admin essentials: owner references and finalizers, CRDs and Operators, kubeadm, one-minor-at-a-time upgrades, etcd snapshots.

Class 11 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. What is Helm and why is it used?

Helm is the package manager for Kubernetes. It packages related manifests as a chart with templates and default values, so one chart can be installed in many environments with different settings, and it tracks releases so you can upgrade and roll back with one command.

Q2. Explain chart, values, release, revision and repository.

A chart is the package; values are the configuration that fills its templates; a release is one installed copy of a chart in a namespace; a revision is a numbered version of that release created by each install, upgrade or rollback; a repository or OCI registry stores published charts.

Q3. What changed between Helm 2 and Helm 3?

Helm 3 removed Tiller, the in-cluster server component, so Helm now uses the user's own kubeconfig and RBAC permissions. Release data is stored as Secrets in the release namespace, chart apiVersion became v2, and OCI registry support was added.

Q4. Describe the structure of a Helm chart.

Chart.yaml (metadata, versions, dependencies), values.yaml (defaults), an optional values.schema.json, a charts/ folder for subcharts, a templates/ folder with manifests, _helpers.tpl, NOTES.txt and tests/, and a .helmignore file.

Q5. What is the difference between version and appVersion in Chart.yaml?

version is the SemVer version of the chart itself and must change whenever the chart changes. appVersion is the version of the application the chart deploys and is informational - often used as the default image tag.

Q6. How do you override values when installing a chart?

With `-f my-values.yaml` (several files allowed; the last wins) or `--set key=value`, `--set-string` and `--set-file`. Priority from lowest to highest: chart values.yaml, parent chart values, -f files, --set.

Q7. What is the difference between helm install, helm upgrade and helm upgrade --install?

install creates a new release and fails if it exists; upgrade changes an existing release and fails if it does not exist; upgrade --install does whichever is needed, which makes it ideal for CI/CD pipelines.

Q8. How do you roll back a Helm release?

Use `helm history <release>` to find the revision, then `helm rollback <release> <revision>`. The rollback itself becomes a new revision. `--atomic` on upgrade rolls back automatically on failure.

Q9. Where does Helm store release information?

In Secrets of type helm.sh/release.v1 in the release's namespace, named like sh.helm.release.v1.<release>.v<revision>. They contain the rendered manifests and values used.

Q10. What is _helpers.tpl?

A file of named templates defined with `define` and used with `include` - typically the full name, common labels and selector labels - so they are written once and reused in every template. Files starting with an underscore do not produce manifests.

Q11. What is the difference between include and template?

Both render a named template, but include returns a string that can be piped into functions such as `nindent` or `quote`, so include is preferred.

Q12. How do you debug a Helm chart before installing it?

Run `helm lint`, render locally with `helm template`, validate against the cluster with `helm install --dry-run --debug`, and preview upgrades with the helm-diff plugin.

Q13. What are Helm hooks? Give an example.

Resources annotated with `helm.sh/hook` that run at lifecycle points such as pre-install, pre-upgrade, post-install, pre-delete or test. A common example is a pre-upgrade Job that runs database migrations before new pods start. Weights order hooks and delete policies clean them up.

Q14. How do you test a Helm release?

Put pods with the annotation `helm.sh/hook: test` in templates/tests and run `helm test <release>`. The test passes if the pods complete successfully.

Q15. How do chart dependencies work?

Dependencies are listed in Chart.yaml with name, version, repository and optional condition. `helm dependency update` downloads them into charts/. The parent sets subchart values under the subchart's name, and `global` values are shared with all subcharts.

Q16. How do you share a chart with other teams?

Package it with `helm package` and push it to an OCI registry with `helm push` after `helm registry login`, or publish to a classic chart repository such as GitHub Pages, ChartMuseum or Harbor.

Q17. How do you make pods restart when a ConfigMap rendered by Helm changes?

Add an annotation to the pod template containing a checksum of the rendered ConfigMap, e.g. `checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}`. Any change alters the template and triggers a rolling update.

Q18. Helm vs Kustomize - when would you use each?

Helm suits distributing and versioning apps, third-party software and charts with many options, and offers hooks, tests and rollback. Kustomize suits your own apps with small per-environment differences using plain YAML overlays, built into kubectl. Many teams use both.

Q19. What are finalizers and owner references?

Owner references link dependents to owners (Pod to ReplicaSet to Deployment) so the garbage collector deletes dependents when the owner is deleted. Finalizers block deletion of an object until a controller completes cleanup; if the controller is missing the object stays Terminating.

Q20. How do you back up and restore etcd?

Take a snapshot with `etcdctl snapshot save` using the etcd endpoint and its CA, cert and key; check it with `etcdutl snapshot status`; restore with `etcdutl snapshot restore --data-dir <new dir>` and point the etcd static pod manifest to the new directory. Use Velero for application volumes.